Hochschule Karlsruhe
University of
Applied Sciences

Aufgabenstellung MSYS

Embedded System Control

Inhaltsverzeichnis

1 Aufgabenstellung **1**
1.1 Implementierung der Multithread-Umgebung 1
1.2 Implementierung des Timings 1
1.3 Ansteuerung der Hardware 2
1.4 Implementierung der Datenübertragung zum Entwicklungsrechner 2
1.5 Kalibrierung der Sensordaten 2
1.6 Implementierung der Zustandsschätzung und Filterung 2
1.7 Reglerauslegung und Simulation mit Simulink 2
1.8 Implementierung der Regelschleife 3

2 Setup Entwicklungsumgebung **4**
2.1 VSCode-Umgebung 4
2.1.1 Verbindung zum Server aufbauen 5
2.1.2 Steckdose anschalten 5
2.1.3 Zugriff auf Würfel 5
2.1.4 Bedienung Würfel zum Balancieren 6

3 Software **7**
3.1 Architektur Target-Anwendung 7
3.2 Comm Comp 9
3.3 Controll Comp 10
3.3.1 Multithread 11
3.3.2 Timing 11
3.3.3 Ansteuerung Hardware (Sensorik, Aktorik) 11
3.3.4 Bereitstellen der Daten 12
3.3.5 Kalibrierung Sensordaten 12
3.3.6 Zustandsschätzung 13
3.3.7 Komplementärfilter 13
3.3.8 Regelung 13

4 Sensoren und Sensorkalibrierung **14**
4.1 Kalibrierung 14
4.1.1 Kalibrierung des AD-Wandlers der Schwungmasse 15
4.1.2 Kalibrierung der Accelerometer 15
4.1.3 Kalibrierung der Gyroskope 16
4.2 Zustandsschätzung und Komplementärfilter 16

5 Reglerauslegung und Simulation **17**

Abbildungsverzeichnis **18**

Quelltextverzeichnis **19**

1 Aufgabenstellung

Das Ziel des Labors ist es, den Würfel in der instabilen Ruhelage zu stabilisieren. Dafür soll das in der Vorlesung erarbeitete Modell genutzt werden, um einen Zustandsregler zu entwerfen, zu simulieren und auf dem Mikrocontroller umzusetzen. Dazu muss zuerst die Entwicklungsumgebung, wie in Kapitel 2 beschrieben, eingerichtet werden. Anschließend kann die schrittweise Implementierung der Target-Anwendung auf dem BeagleBone durchgeführt werden. Auch gibt es bereits Klassen zum Speichern aller möglichen Daten (Imu-Daten, Zustandsvektor etc.). Verwenden Sie diese und machen Sie sich mit dem Framework vertraut, indem Sie sich anschauen, welche Klassen existieren! Das empfohlene Vorgehen zur Realisierung der Regelung des Würfels wird im folgenden Abschnitt beschrieben. Details und Hilfestellungen zur Implementierung der einzelnen Aufgaben finden sich in Kapitel 3. Machen Sie sich Gedanken über wiederverwendbaren, strukturierten und leserlichen Code.

1.1 Implementierung der Multithread-Umgebung

Implementieren Sie ein Multithread-Programm in dem zwei Threads erzeugt werden, die gleichzeitig ausgeführt werden. Dazu soll das *IRunnable*-Interface des Frameworks verwendet. Die Threads bieten die Grundlage für den späteren Producer-Consumer-Austausch zwischen dem Thread zur Regelung (ControlComp) und dem Thread zum Versenden der Daten des Würfels (CommComp). Achten Sie darauf, einen geeigneten Mechanismus zum Beenden der beiden gestarteten Threads zu implementieren, um das Programm per Tastatureingabe stoppen zu können.

1.2 Implementierung des Timings

Um das feste Abtastintervall garantieren zu können, muss ein geeigneter Mechanismus zum Timing des zeitgebenden Threads (ControlComp) implementiert werden. Starten Sie hierfür nur einen Thread, die CommComp ist für diese Aufgabe nicht benötigt. Sobald Sie die Implementierung der Datenübertragung (siehe Abschnitt 1.4) abgeschlossen haben, sollen Sie den mitgeschickten Zeitstempel (zu finden in der csv Datei, die automatisch beim Ausführen der GUI erstellt wird) auswerten um den Jitter des Systems zu bestimmen. Beurteilen Sie, ob die Genauigkeit Ihres Timers ausreicht.

1.3 Ansteuerung der Hardware

Implementieren Sie den Zugriff auf die Sensorik und Aktorik über die bereitgestellte Hardware-Klasse. Dies umfasst das Auslesen der IMU-Rohwerte sowie das Aktivieren und Ansteuern des Motors über ein definiertes Drehmoment. Die Zugriffe werden über das vorhandene Framework verwaltet und sollen in der ControlComp geschehen.

1.4 Implementierung der Datenübertragung zum Entwicklungsrechner

Realisieren Sie eine thread-sichere Datenübergabe zwischen der Control-Comp (Producer) und der Comm-Comp (Consumer). Nutzen Sie zur Synchronisation einen Semaphoren-Mechanismus, um die Daten verlustfrei an die Python-GUI zu übertragen. Schauen Sie sich an, welche Daten von der Übertragung erwartet werden. Übertragen Sie anstelle der Zustandsdaten zunächst Dummy-Daten, um die Funktion zu überprüfen. Testen Sie Ihren Timer über den mitzuschickenden Zeitstempel. Machen Sie sich Gedanken über das sichere und sinnvolle Starten und Beenden der Producer-Consumer-Kommunikation.

1.5 Kalibrierung der Sensordaten

Ermitteln Sie die Kalibrierparameter (Scale und Offset) für alle Sensoren durch Referenzmessungen. Überführen Sie die digitalen Rohwerte der Accelerometer, Gyroskope und des AD-Wandlers mithilfe linearer Polynome in physikalische SI-Einheiten. Nutzen Sie die Datenübertragung zur Python-Gui um die Referenzmessungen auf Ihrem PC verarbeiten zu können.

1.6 Implementierung der Zustandsschätzung und Filterung

Berechnen Sie den aktuellen Systemzustand aus den kalibrierten Sensordaten. Implementieren Sie zur Fusion der IMU-Daten einen Komplementärfilter, um einen driftfreien und rauscharmen Schätzwert für den Würfelwinkel zu erhalten. Validieren Sie Ihre Ergebnisse durch das Bewegen des Würfels und die Live-Datenanzeige in der Python-Gui.

1.7 Reglerauslegung und Simulation mit Simulink

Entwerfen Sie einen zeitdiskreten Zustandsregler nach dem LQR-Verfahren. Validieren Sie das Regelkonzept in der Simulation am nichtlinearen Modell und optimieren Sie die Gewichtungsmatrizen Q und R auf Basis der geforderten Performance-Vorgaben.

1.8 Implementierung der Regelschleife

Überführen Sie die ermittelte Reglermatrix K in die Target-Anwendung. Implementieren Sie die zyklische Berechnung der Stellgröße, um den Würfel im instabilen Gleichgewichtspunkt zu balancieren. Wenn Sie alles richtig gemacht haben, steht Ihr Würfel nun - Glückwunsch! Bedenken Sie den Würfel in die richtige Ausgangslage zu bringen, bevor Sie ihr Experiment starten.

2 Setup Entwicklungsumgebung

Momentan werden zwei Entwicklungsumgebungen angeboten. Die alte Variante basiert auf Eclipse, was in einer virtuellen Box ausgeführt wird. Allerdings wird diese Umgebung nicht mehr aktiv gepflegt und ist an vielen Stellen nicht nach neuem Stand der Technik umgesetzt. Eine ausführliche Anleitung mit allen Links findet sich im Ilias unter Legacy. Als Alternative kann VSCode verwendet werden. Dafür ist ebenfalls eine virtuelle Box verfügbar, die Umgebung kann aber mit wenig Aufwand auf allen Linux-Geräten (vorzugsweise Ubuntu) aufgesetzt werden (natives Linux, virtuelle Maschine, WSL). Eine ausführliche Anleitung dazu findet sich im Github-Repo mit der Umgebung `https://github.com/ruben01egle/BBBCube`. Grundsätzlich ist das VSCode-Setup schneller, einfacher zu bedienen, bietet mehr Funktionen und nutzt einen moderneren Workflow.

2.1 VSCode-Umgebung

Die Nutzung von Git ist sehr zu empfehlen. Um das Repository herunterzuladen, folgenden Befehl in ein Terminal an einem geeigneten Speicherort eingeben:

```
1 git clone https://github.com/ruben01egle/BBBCube
```

Listing 1: Clone VSCode-Repository

Danach kann der Ordner *BBBCube* mit VSCode geöffnet werden. Eine ausführliche Anleitung zur Installation aller Pakete und eine Übersicht aller Funktionen und deren Verwendung findet sich in der *BBBCube/README*-Datei. Diese Datei sollte gründlich durchgelesen werden, im Folgenden wird nur eine kurze Zusammenfassung dargestellt. Eine Übersicht zu VSCode ist in Abbildung 2.1 zu sehen.

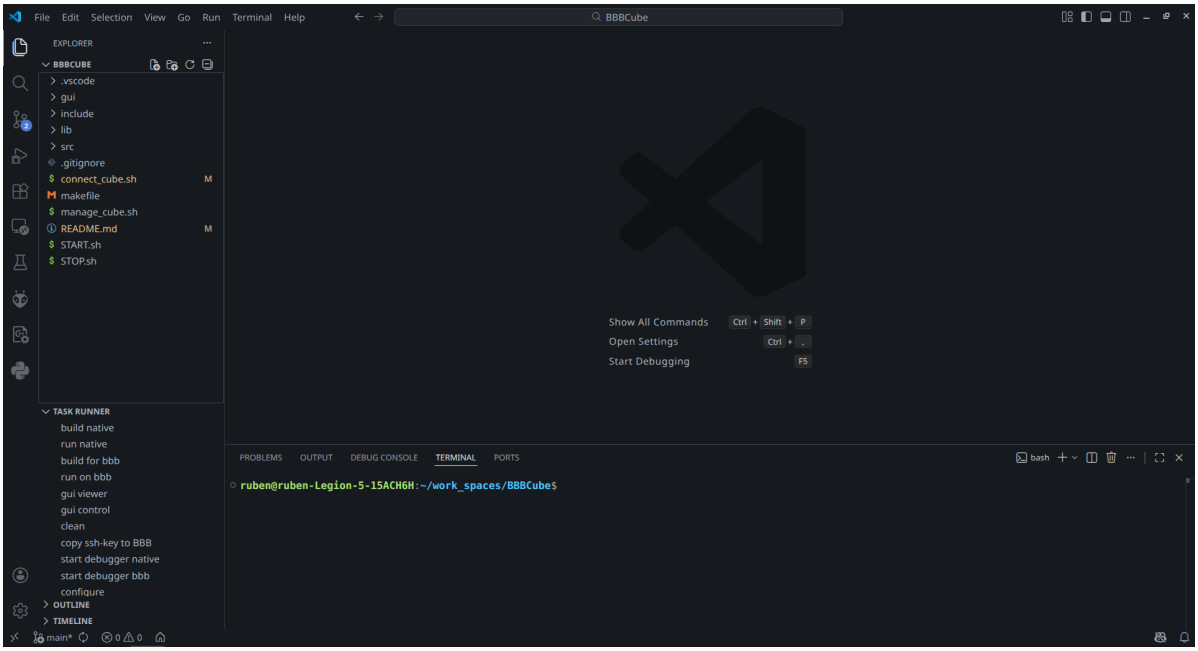

Abbildung 2.1: VSCode Oberfläche

2.1.1 Verbindung zum Server aufbauen

Verbindung zum Server aufbauen, der alle Würfel verwaltet. Dazu wird folgender Befehl im Terminal ausgeführt.

```
./connect_cube.sh start groupX
```

Listing 2: Aufbau zum Server der Würfel

2.1.2 Steckdose anschalten

Steckdose für die Würfel anschalten. Dazu `http://localhost:42000` aufrufen und einschalten.

2.1.3 Zugriff auf Würfel

Die Verbindung zu den beiden BeagleBones und der Kamera zur Steuerung des Würfels ist nun möglich. Der Zugriff auf das BeagleBone zur Regelung des Würfels kann per SSH erfolgen:

```
ssh root@localhost -p48000
```

Listing 3: Aufbau SSH Verbindung zum Würfel

Auf die Kamera und das BeagleBone zum Hochfahren des Würfels kann mit dem Python-Skript *gui/cube_control.py* zugegriffen werden. Diese Funktion ist im VSCode-Task *gui control* automatisiert. Um das Projekt zu bauen, auf dem eigenen PC oder auf dem BeagleBone laufen zu lassen gibt es ebenfalls VSCode-Tasks:

- *gui control*: Startet die Anwendung für die Kamera und zum Hochfahren des Würfels
- *built native*: Kompiliert das Projekt für den lokalen PC
- *run native*: Führt das Programm auf dem lokalen PC aus
- *build for bbb*: Kompiliert das Projekt für das BeagleBone
- *run on bbb*: Führt das Programm auf dem BeagleBone aus
- *gui viewer*: Plottet und speichert vom BeagleBone geschickten Datenstream für Sensordaten etc. !!Funktioniert erst wenn entsprechendes Programm auf dem BeagleBone auch Daten verschickt!!

Eine detaillierte Übersicht aller Funktionen ist in *BBBCube/README* zu finden.

2.1.4 Bedienung Würfel zum Balancieren

Wenn die Target-Anwendung implementiert ist und das Balancieren getestet werden soll, muss der Würfel zuerst in die aufrechte Lage gebracht werden. Nun wird der Würfel aber festgehalten, und kann somit nicht regeln und wird unweigerlich umfallen, sobald er losgelassen wird. Daher muss er erst auf 5 Grad Gap gestellt werden. Dann hat der Würfel die Freiheit, sich selbst aufzurichten und zu regeln. Nun kann die Target-Anwendung gestartet werden. Sobald sich der Würfel aufgestellt hat und sicher balanciert, kann er released werden.

3 Software

Das folgende Kapitel beschreibt die Architektur der Anwendung zum Balancieren des Würfels. Fortgeschrittene Teams können eigenständig die Komponenten implementieren. Bei Schwierigkeiten können die Hilfestellungen mit mehr Anweisungen am Ende der Beschreibung der jeweiligen Komponente zur Hilfe herangezogen werden. Prinzipiell sind bereits sehr viele Klassen im Framework implementiert. Die Ansteuerung der Hardware und die Verwaltung der Threads kann direkt so verwendet werden. Auch gibt es bereits Klassen zum Speichern aller möglichen Daten (Imu-Daten, Zustandsvektor etc.). Verwenden Sie diese und machen Sie sich mit dem Framework vertraut, indem Sie sich anschauen, welche Klassen existieren!

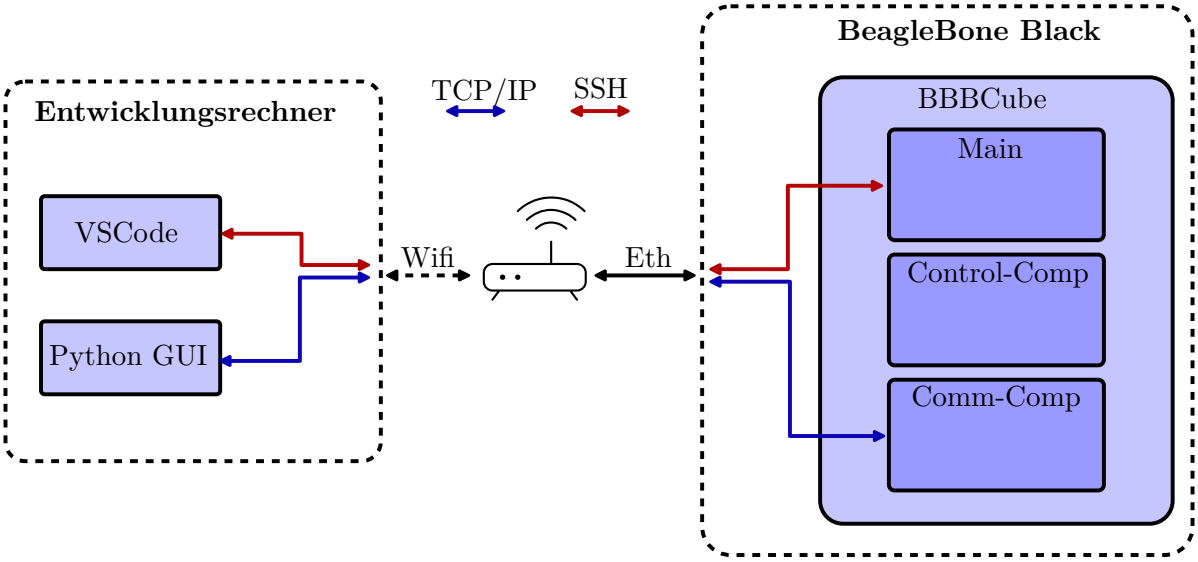

Abbildung 3.1: Übersicht Software

3.1 Architektur Target-Anwendung

Die Architektur und der Ablauf der Targetanwendung ist in Abbildung 3.2 dargestellt. Diese besteht aus drei Komponenten, die jeweils in einem eigenen Thread laufen. Die Main-Komponente verwaltet die anderen beiden Komponenten und verarbeitet Nutzereingaben. Zusätzlich werden die beiden anderen Komponenten erstellt und gestartet. Die Control-Comp kümmert sich um die Umsetzung der Regelkreise und die Ansteuerung der Hardware, während die Comm-Comp Daten des Würfels an die Python-Gui auf dem Entwicklungsrechner verschickt.

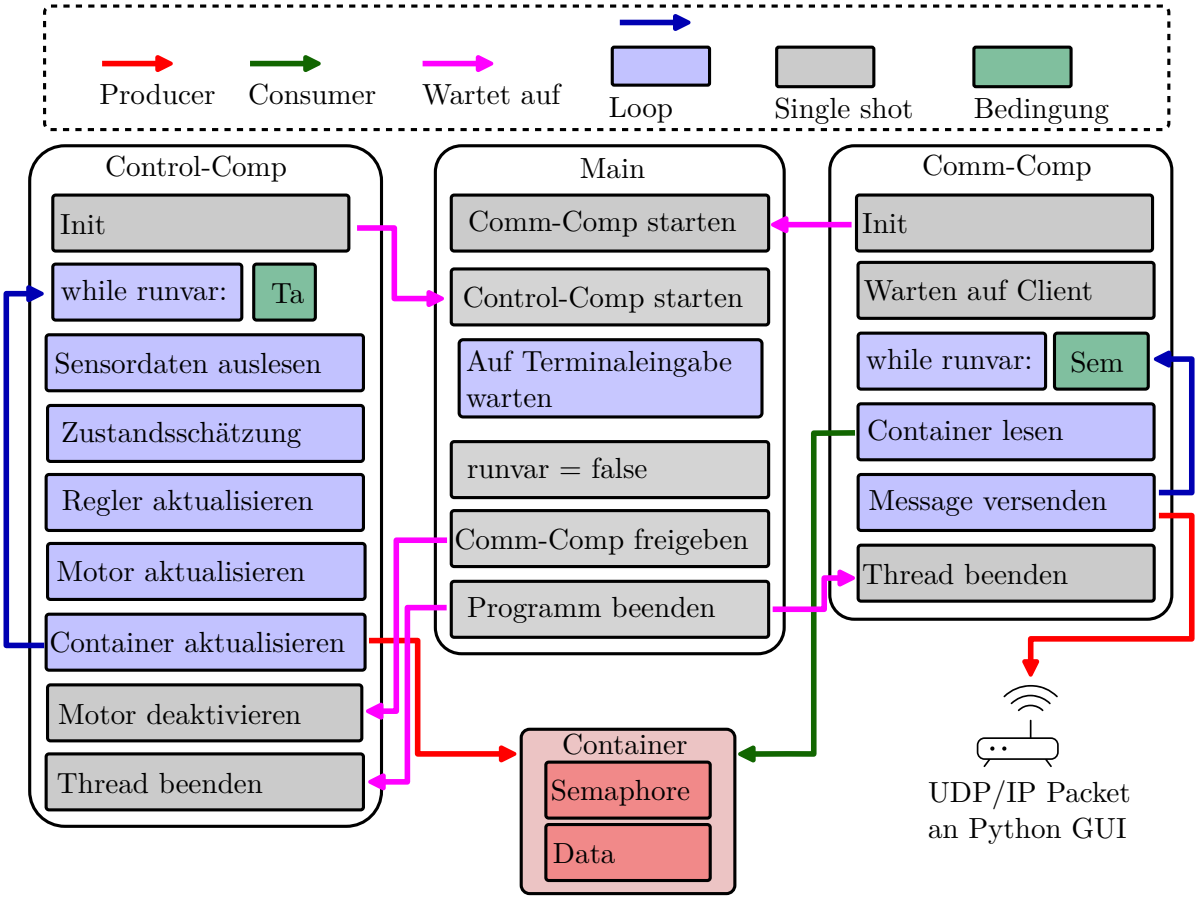

Abbildung 3.2: Softwarearchtitektur

Die Übertragung der Daten von der Control-Comp, die als Producer agiert, zur Comm-Comp, die die Rolle des Consumers übernimmt, erfolgt über einen globalen Container. Da beide Threads auf den Container zugreifen können, ist ein geeigneter Mechanismus zur Synchronisierung zu implementieren.

Um alle Threads sicher zu beenden, muss die Main-Komponente den anderen Threads miteilen, dass diese sich beenden sollen. Geschiet dies nicht, bleiben die Threads im Hintergrund offen, blockieren erneutes Ausführen und Destruktoren werden nicht aufgerufen. Das führt zu undefinierten Hardwarezuständen (z.B. Motor gibt Vollgas) und zu Speicherlecks. Daher muss ein geeigneter Mechanismus implementiert werden, der sicherstellt, dass alle Threads ordnungsgemäß beendet werden.

Da es sich beim Mikrocontroller um ein diskretes System handelt, müssen strenge Timing-Anforderungen eingehalten werden. Diese spielen bei der Architektur der Anwendung eine maßgebliche Rolle. Regelkreise stellen strenge Echtzeitanforderungen an den Mikrocontroller, da die Datenerfassung, Berechnung und Ansteuerung der Aktoren in definierten Zeitintervallen

erfolgen müssen. Kommt es zu Verzögerungen oder variablen Latenzen (Jitter), verändert sich das effektive Abtastintervall, was die Stabilität des Reglers gefährdet, Schwingungen verstärken oder zum Kontrollverlust des Würfels führen kann. Linux ist aber nicht echtzeitfähig, weil das Scheduling und die Verarbeitung von Interrupts keinem deterministischen Timing folgt. Daher bringt jeder Kontextwechsel zusätzliche, nicht deterministische Latenz mit sich. Darum sollten Kontextwechsel so gut es geht vermieden werden. Aus diesem Grund werden alle Aufgaben, die Teil des Regelkreises sind, in einem Thread (in der Control-Comp) hintereinander ausgeführt. Somit gibt es einen Kontextwechsel nur, wenn die Control-Comp bis zum nächsten Abtastintervall in den Sleep-Zustand übergeht. Währenddessen kann die Comm-Comp die Fahrzeugdaten per TCP versenden. Diese Aufteilung wird gewählt, um die Echtzeitanforderungen des Systems möglichst genau einhalten zu können.

In der gesamten Software gibt es zahlreiche Parameter, von den Reglerparametern über die Parametrierung der Hardware bis hin zu Filterkoeffizienten, die angepasst werden können. Um eine einheitliche, übersichtliche Schnittstelle zu schaffen, wird in der Datei *CubeConstants.hpp* ein Namespace *Cube* angelegt. Alle Parameter, die nicht fest vorgegeben sind (z.B. Speicheradressen bestimmter Register), werden in diesem Namespace gespeichert und können an den entsprechenden Stellen eingebunden werden.

3.2 Comm Comp

Die Comm-Comp hat die Aufgabe, die Daten des Würfels per TCP/IP an die Python-Gui auf dem Entwicklungsrechner zu schicken. Der Code für die Client-Server-Verbindung zwischen Python-Gui und Comm-Comp ist im Framework schon implementiert. Nach der Initialisierung des Servers soll die Comm-Comp auf eine eingehende Verbindung der Python-Gui warten. Wird diese hergestellt, soll die Comm-Comp, immer wenn neue Daten im globalen Container vorhanden sind, diese Daten lesen und verschicken. Dafür muss die Comm-Comp als Consumer einen geeigneten Mechanismus zur Synchronisierung mit der Controll-Comp (dem Producer) haben.
Die relevanten Funktionen für den Aufbau der Verbindung und das Verschicken der Daten sind in der Klasse *CServer* zu finden:

void init(): Diese Funktion initialisiert den Server.

bool waitForClient(): Diese Funktion blockiert den Thread, in dem sie aufgerufen wurde, solange, bis eine eingehende Verbindung erkannt wird. Die Funktion gibt eine Statusmeldung über den Erfolg der aufzubauenden Verbindung zurück.

bool transmitMessage(SContent& content): Diese Funktion verschickt eine Nachricht per TCP an die im Server gespeicherte Verbindung. Der Inhalt der Nachricht wird durch die Datenklasse *SContent* übergeben. Der Rückgabewert zeigt an, ob die Nachricht verschickt werden konnte.

Es ist dabei zu beachten, wann welche Komponente gestartet wird. Wird die Controll-Comp gestartet bevor die Verbindung zwischen der Comm-Comp und der Python-Gui hergestellt

wurde, gehen die Daten in diesem Zeitraum verloren. Für die Beurteilung der Regelung sind aber genau die ersten Millisekunden nach dem Start der Controll-Comp relevant.

Zusätzliche Hinweise Producer-Consumer Da bei der Übertragung der Daten zwischen Controll-Comp und Comm-Comp die Daten nur einmal gelesen werden dürfen, werden Semaphoren eingesetzt. Normalerweise ist in so einem Kontext ein Cross-Handshake notwendig. Dabei signalisiert die Control-Comp das Vorhandensein neuer Daten per Semaphore, und die Comm-Comp meldet, wenn die Verarbeitung fertig ist. So wird garantiert, dass Daten weder mehrfach, gleichzeitig noch teilweise verarbeitet werden. Allerdings ist der Consumer in diesem Fall garantiert schneller als der Producer. Außerdem darf die Control-Comp unter keinen Umständen blockiert werden, da dies die Stabilität des Fahrzeugs gefährdet. Aus diesen Gründen ist eine einfache Semaphore, die der Comm-Comp signalisiert, wenn neue Daten vorhanden sind, ausreichend.

3.3 Controll Comp

Die Controll-Comp ist für die Regelung des Würfels verantwortlich. Damit unterliegen ihr folgende Aufgaben:

- Timing
- Sensorik auslesen
- Sensordaten kalibrieren
- Zustandsschätzung
- Komplementärfilter
- Berechnung Stellgröße
- Ansteuerung der Aktorik
- Daten zum Auslesen bereitstellen

Alle diese Aufgaben werden in Module ausgelagert. Teilweise sind diese schon implementiert (z.B. Ansteuerung der Hardware) oder müssen noch von Ihnen implementiert werden (z.B. die Zustandsschätzung). Der folgende Abschnitt beschreibt die Aufgaben der einzelnen Module und gibt Hinweise zu deren Implementierung. Unter zusätzliche Hinweise sind Tipps zu finden, wenn Unklarheiten oder Probleme bei der Implementierung auftreten.

3.3.1 Multithread

Die Regelung des *Monowheelers* und der Datenaustausch mit der Python Gui sind auf zwei Komponenten (Controll-Comp und Comm-Comp) aufgeteilt. Jede Komponente soll in einem eigenen Thread gestartet werden. Die Verwaltung dieses Threads wird von der Klasse *CThread* übernommen. Die Klasse *CThread* erwartet ein Objekt mit dem Interface *IRunnable*, eine virtuelle Klasse die eine Init()- und eine Run()-Funktion bereitstellt. Wird die start()-Funktion von *CThread* aufgerufen, wird ein neuer Thread gestartet, der zuerst die Init()- Funktion und dann die Run()-Funktion des *IRunnable*-Interfaces ausgeführt. Die Controll-Comp und Comm-Comp sollen also Klassen sein, die von der virtuellen Klasse *IRunnable* erben, sodass sie über *CThread* verwaltet werden können. Dadurch können beispielsweise auch verschiedene Prioritäten der Threads oder die Scheduling-Methode festgelegt werden können.

Zusätzliche Hinweise Beenden des Programms Wird die Eingabe zum Stoppen des Programms getätigt, wird von der Main den anderen Komponenten mit einer globalen Variable *runvar* signalisiert, sich zu beenden. Das ist eine sehr simple und zuverlässige Variante, um einfach Daten zwischen zwei Threads auszutauschen. Dabei muss trotzdem beachtet werden, dass es sich um eine Shared-Ressource handelt. Das kann bei der Compiler-Optimierung und beim gleichzeitigen Zugriff zu Problemen führen. Idealerweise sollte daher eine atomic-Flag verwendet werden. Die Control-Komponente beendet sich nach dem nächsten Abtastschritt, wenn die Flag gesetzt wurde. Um sicherzustellen, dass die Kommunikations-Komponente nicht auf der Semaphore blockiert ist, gibt die Main die Semaphore erneut frei. Haben sich sowohl die Control-Comp als auch die Comm-Comp beendet, wird das Programm beendet.

3.3.2 Timing

Auf klassischen Mikrocontrollern ohne Betriebssystem können Hardware-Timer zur Zeitmessung verwendet werden. Dadurch ist harte Echtzeit möglich. Auf einem Linux-System wie dem BBB werden die Hardware-Interrupts allerdings vom Linux-Kernel verwaltet. Daher können Posix-Funktionen im Userspace verwendet werden, um auf die Hardware-Timer zuzugreifen. Damit kann ein Timer aufgerufen werden, der die aktuelle Zeit auf Mikrosekunden genau ausgibt (z.B. mit *clock_gettime()*). Damit kann der Zeitstempel zu Beginn der Schleife gespeichert werden. Nach dem Schleifendurchlauf kann der aktuelle Zeitstempel erneut gemessen werden. Durch die Differenz zwischen Start- und Endzeit lässt sich bestimmen, wie viel Zeit bis zum nächsten Abtastintervall gewartet werden muss. Diese Zeit kann der Thread dann mit einer Funktion wie *clock_nanosleep()* schlafen, um der Comm-Comp Zeit zum Ausführen zu geben.

3.3.3 Ansteuerung Hardware (Sensorik, Aktorik)

Die Implementierung der Ansteuerung der Hardware ist bereits im Framework in *lib/Hardware* implementiert. Der Zugriff auf die Hardware erfolgt durch die Klasse *CBBBHardware*. Dort werden die folgenden Funktionen implementiert:

bool fetchValues(UInt16& adcValue, CIMUData& sensor1Data, CIMUData& sensor2Data): Diese Funktion liest die Rohwerte der Sensoren aus. Dazu zählen die beiden IMUs zur Lagebestimmung sowie der ADC zur Bestimmung der Geschwindigkeit der Schwungscheibe. Die Rohdaten werden in die übergebenen Datenobjekte der Typen *UInt16* und *CIMUData* geschrieben. Die Funktion liefert eine Statusmeldung über den Erfolg des Aufrufs als Rückgabewert.

bool enableMotor()/bool disableMotor(): Schaltet den Motor Ein/Aus. Muss bei Beginn und Ende des Programms einmalig ausgeführt werden. Die Funktion liefert eine Statusmeldung über den Erfolg des Aufrufs als Rückgabewert.

bool setTorque(float torque): Ist der Motor aktiviert, setzt diese Funktion das übergebene Drehmoment am Motor. Die Funktion liefert eine Statusmeldung über den Erfolg des Aufrufs als Rückgabewert.

3.3.4 Bereitstellen der Daten

Die Daten, die an den Entwicklungsrechner geschickt werden sollen, werden mit dem globalen Container an die Comm-Comp (Consumer) übergeben. Die Controll-Comp (Producer) muss also bei jedem Durchlauf die Daten im Container aktualisieren. Um die Datenübertragung threadsicher zu gestalten und Race-Conditions zu vermeiden, enthält der Container neben den Daten einen Mechanismus zur Synchronisierung. Die Controll-Comp muss dafür den Producer-Teil des Mechanismus zur Synchronisierung implementieren.

Zusätzliche Hinweise Producer-Consumer Da bei der Übertragung der Daten zwischen Controll-Comp und Comm-Comp die Daten nur einmal gelesen werden dürfen, werden Semaphoren eingesetzt. Normalerweise ist in so einem Kontext ein Cross-Handshake notwendig. Dabei signalisiert die Control-Comp das Vorhandensein neuer Daten per Semaphore, und die Comm-Comp meldet, wenn die Verarbeitung fertig ist. So wird garantiert, dass Daten weder mehrfach, gleichzeitig noch teilweise verarbeitet werden. Allerdings ist der Consumer in diesem Fall garantiert schneller als der Producer. Außerdem darf die Control-Comp unter keinen Umständen blockiert werden, da dies die Stabilität des Fahrzeugs gefährdet. Aus diesen Gründen ist eine einfache Semaphore, die der Comm-Comp signalisiert, wenn neue Daten vorhanden sind, ausreichend.

3.3.5 Kalibrierung Sensordaten

Die rohen Sensordaten aller Sensoren müssen mit dem Kalibrierpolynom wie in Kapitel 4 beschrieben kalibriert werden. Die konstanten Faktoren der Polynome sollen dabei in der Datei *CubeConstants.hpp* gespeichert werden. Bei der Berechnung der physikalischen Sensorwerte müssen sowohl Einheiten als auch die Datentypen der Rohwerte und physikalischen Werte beachtet werden.

3.3.6 Zustandsschätzung

Nicht alle relevanten Daten des Würfels ergeben sich direkt aus den Sensordaten. Daher müssen die gewünschten Zustände aus den Sensordaten berechnet werden. Da die Zustände teilweise nicht direkt gemessen werden, wird dieser Prozess Zustandsschätzung genannt. Die relevanten Größen sind die Einträge des Zustandsvektors: der Winkel des Würfels, die Winkelgeschwindigkeit und die Geschwindigkeit der Schwungscheibe. Grundlage hierfür sind Messdaten der beiden IMUs, die sich in bekannten Abständen zum Drehpunkt befinden, sowie der ADC-Wandler zur Bestimmung der Drehzahl der Schwungscheibe. Der Abstand der ersten IMU beträgt $r_1 = 0{,}14\,\mathrm{m}$, der Abstand der zweiten IMU $r_2 = 0{,}061\,\mathrm{m}$.

Auf Basis der kalibrierten Sensordaten sollen geeignete Schätzwerte für die Zustandsgrößen berechnet und zusammengefasst ausgegeben werden. Dabei sind folgende physikalische Zusammenhänge zu berücksichtigen:

1. Die Winkelgeschwindigkeit der Schwungmasse $\dot{\psi}$ ergibt sich direkt aus der entsprechenden kalibrierten Messung.
2. Die Winkelgeschwindigkeit des Würfels $\dot{\varphi}$ wird aus den Messwerten der beiden Gyroskope bestimmt, indem deren Beiträge geeignet kombiniert werden (z. B. durch Mittelung).
3. Der geschätzte Winkel des Würfels φ_A wird aus den gemessenen linearen Beschleunigungen der beiden IMUs berechnet. Dabei gilt

$$\varphi_A = \arctan\left(\frac{\ddot{x}_1 - \dfrac{r_1}{r_2} \cdot \ddot{x}_2}{\ddot{y}_1 - \dfrac{r_1}{r_2} \cdot \ddot{y}_2}\right),$$

   wobei $\ddot{x}_i$ und $\ddot{y}_i$ die gemessenen Beschleunigungskomponenten der jeweiligen IMU und r_1, r_2 deren Abstände zum Drehpunkt darstellen.

3.3.7 Komplementärfilter

Zur Verbesserung der Schätzung des Würfelwinkels wird ein Komplementärfilter implementiert. Das Filter arbeitet wie der Rest des Regelkreises mit einer festen Abtastperiode von $T = 0{,}02\,\mathrm{s}$ und ist in Abschnitt 4.2 beschrieben.

Als Eingang dienen die zuvor geschätzten Zustandsgrößen, der Ausgang ist der gefilterte Winkel des Würfels. Für den ersten Abtastschritt wird der aktuell geschätzte Winkel direkt als Filterausgang verwendet. Für alle weiteren Abtastschritte wird der gefilterte Winkel rekursiv berechnet.

3.3.8 Regelung

Der Eingang der Regelung ist der Zustandsvektor. Zur Bestimmung der Stellgröße (Motormoment) wird der Zustandsvektor mit der Reglermatrix K zeilenweise multipliziert.

4 Sensoren und Sensorkalibrierung

Die Sensoren des Würfels liefern Informationen über die physikalischen Zustände des Würfels. Um aus den digitalen Sensorrohwerten die physikalischen Zustände herauszufinden, sind verschiedene Schritte notwendig.

4.1 Kalibrierung

Sensoren müssen kalibriert werden, weil ihre rohen Messwerte nicht direkt den realen physikalischen Größen entsprechen. Kalibrierung stellt sicher, dass Messungen korrekt, vergleichbar und nutzbar sind. Die wichtigsten Gründe sind:

- Kein Sensor ist exakt gleich. Durch Fertigungsstreuungen unterscheiden sich die Rohdaten bei gleicher Messung zwischen Sensoren.
- Sensoren liefern meist digitale Rohwerte (z.,B. ADC-Zählwerte). Erst durch Kalibrierung lassen sich diese sinnvoll in SI-Einheiten umrechnen.
- Korrektur von Offsets (Nullpunktfehlern). Viele Sensoren zeigen auch dann einen Wert an, wenn die gemessene Größe eigentlich null ist (z.,B. Gyroskope im Stillstand). Die Kalibrierung entfernt diesen systematischen Fehler.

Ziel dieser Aufgabe ist die Kalibrierung aller im System verwendeten Sensoren, sodass deren digitale Messwerte in physikalische Größen in SI-Einheiten überführt werden können. Für jeden Sensor wird ein lineares Kalibrierpolynom aufgestellt:

$$data = scale \cdot (data_{raw} - offset)$$

Damit müssen für jeden Sensor zwei Faktoren bestimmt werden. Pro Sensor sind daher mindestens zwei Messpunkte erforderlich. Die Skalierungsfaktoren können teilweise aus Datenblättern entnommen werden, sodass in diesen Fällen nur noch eine Messung zur Bestimmung des Offsets notwendig ist.

Zur Bestimmung der Kalibrierfaktoren eines Sensors werden Messungen unter definierten Bedingungen durchgeführt, bei denen der zu messende physikalische Wert bekannt ist. Für jede dieser Referenzsituationen wird der digitale Rohwert des Sensors aufgezeichnet. Durch den Vergleich des bekannten physikalischen Werts mit dem zugehörigen digitalen Messwert entsteht ein Datenpaar, das den Zusammenhang zwischen Sensorrohwert und physikalischer Größe beschreibt.

Werden zwei solche Datenpaare aufgenommen, so ergeben sich zwei lineare Gleichungen mit den

beiden unbekannten Parametern *scale* und $offset$. Der physikalische Wert *data* ist für beide Messpunkte bekannt, da die physikalische Größe, die gemessen wird, bekannt ist. $data_{raw}$ wird vom Sensor geliefert. Damit entsteht ein lineares Gleichungssystem, das eindeutig gelöst werden kann, um die Kalibrierparameter des Sensors zu bestimmen. Für eine bessere Genauigkeit kann es sinnvoll sein, mehr Datenpunkte aufzunehmen. Das überbestimmte Gleichungssystem kann dann mit Funktionen wie *polyfit* approximiert werden.

Beim Würfel gibt es die Möglichkeit, die Sensorrohwerte vom BeagleBone an die Python-Gui zu schicken. Dort werden die Daten automatisch in eine CSV-Datei geloggt, sodass sie zur Auswertung zur Verfügung stehen. Der Würfel wird also in einen bekannten Zustand gebracht und eine Messung von ca. 10 s aufgezeichnet. Die Sensorrohdaten können dann mit Python oder Matlab eingelesen werden und die Kalibrierparameter bestimmt werden.

4.1.1 Kalibrierung des AD-Wandlers der Schwungmasse

Zur Bestimmung des Kalibrierpolynoms des AD-Wandlers wird eine bekannte Motorkonfiguration genutzt. Der Motortreiber ist so eingestellt, dass bei einer Rotationsgeschwindigkeit von 157 rad/s ein digitaler Messwert von 4096 ausgegeben wird und bei −157 rad/s ein digitaler Messwert von 0. Anhand dieser beiden Referenzpunkte kann das lineare Kalibrierpolynom bestimmt werden, indem die beiden unbekannten Parameter a (Skalierung) und b (Offset) so gewählt werden, dass die bekannten Geschwindigkeiten den gemessenen Rohwerten entsprechen. Zusätzlich sollte noch eine Messung durchgeführt werden, um das Offset des Sensors zu bestimmen. Dazu wird berechnet, welcher Rohwert im Stillstand erwartet wird, und dies mit den tatsächlichen Sensorwerten vergleichen.

4.1.2 Kalibrierung der Accelerometer

Die Kalibrierung der Accelerometer erfolgt experimentell. Hierzu wird der Würfel in mehreren bekannten Ruhelagen positioniert und die digitalen Messwerte der Sensoren in den jeweiligen Messrichtungen aufgezeichnet und gemittelt. Mithilfe der bekannten Zusammenhänge zwischen Würfelwinkel und Gravitationsbeschleunigung

$$\ddot{x}_i = \sin(\varphi)\, g, \qquad \ddot{y}_i = -\cos(\varphi)\, g$$

können den Messwerten die entsprechenden physikalischen Beschleunigungen zugeordnet werden. Aus den resultierenden Wertepaaren wird für jede Achse ein Kalibrierpolynom bestimmt. Zudem ist die Ausrichtung der IMUs (Inertial Measurement Units) zu beachten. Hierfür müssen die Orientierungen der Sensoren durch das Auslesen der Rohwerte in definierten Referenzpositionen bestimmt werden. Dies ist notwendig, um die Sensordaten softwareseitig an das Koordinatensystem des Würfels anzupassen.

4.1.3 Kalibrierung der Gyroskope

Für die Kalibrierung der Gyroskope zur Bestimmung der Winkelgeschwindigkeit des Würfels $\dot{\varphi}$ wird die im Datenblatt angegebene Verstärkung $scale = 0{,}001\,060\,\mathrm{rad\,s^{-1}}$ verwendet. Das Offset wird experimentell bestimmt, indem die Sensorausgänge bei ruhendem Würfel aufgezeichnet und gemittelt werden.

4.2 Zustandsschätzung und Komplementärfilter

Beide Sensortypen in den Imus (Accelerometer und Gyroskope) weisen spezifische Stärken und Schwächen auf. Die Accelerometer messen nicht nur die Erdbeschleunigung, sondern auch die Beschleunigungen durch rotatorische und translatorische Bewegungen, wodurch es zu Fehlern in der Winkelschätzung kommt. Diese Einflüsse können durch die Verwendung von zwei Imus und dem geometrischen Trick teilweise eliminiert werden. Die Accelerometer liefern aber langfristig stabile Werte, da keine Integration erforderlich ist. Die Gyroskope hingegen reagieren sehr gut auf schnelle Bewegungen und erfassen die Dynamik des Fahrzeugs präzise. Dafür sind sie anfällig für Drift, da sich die Integrationsfehler nach und nach aufsummieren.

Um die Zustandsschätzung zu verbessern, wird ein Komplementärfilter eingesetzt. Dabei werden die Sensordaten beider Sensortypen so kombiniert, dass die Stärken der Sensoren genutzt und die Schwächen durch den jeweils anderen Sensor ausgeglichen werden. Dabei wird die Winkelschätzung durch die Gyroskope mit einem Hochpassfilter belegt, sodass der Drift (sehr niedrige Frequenz) gefiltert wird. Die hohen Frequenzen bilden die schnellen Bewegungen des Würfels ab. Die Daten des Accelerometer werden mit einem Tiefpassfilter mit derselben Grenzfrequenz wie der Hochpassfilter der Gyroskope belegt, sodass hier die hochfrequenten Rauschanteile gefiltert werden, aber die langfristige Korrektur der Orientierung genutzt wird. Die Berechnungsvorschrift des Filters sieht wie folgt aus:

$$\varphi_n^C = \alpha \left(\varphi_{n-1}^C + T\,\dot{\varphi} \right) + (1-\alpha)\,\varphi_n^A,$$

wobei n den Abtastindex bezeichnet, φ^C den Wert für den Winkel des Komplementärfilters, φ^A den aus der Zustandsschätzung gewonnenen Winkel und $\dot{\varphi}$ die Winkelgeschwindigkeit des Würfels. Die Winkelgeschwindigkeit der Schwungmasse $\dot{\psi}$ sowie die Winkelgeschwindigkeit des Würfels $\dot{\varphi}$ werden nicht gefiltert, sondern können für die Regelung direkt aus der Zustandsschätzung übernommen werden. Der Faktor α bestimmt die Grenzfrequenz des Hoch- und Tiefpassfilters. Eine geeignete Grenzfrequenz von ca. 0,16 Hz ergibt sich bei einem Gewichtungsfaktor $\alpha = 0{,}98$.

Zu beachten ist zudem, dass der Zustandsfilter mit dem Startwert 0 initialisiert wird. Die Gyroskope können eine anfängliche Schräglage nicht erfassen, da zu Beginn keine Bewegung vorliegt. Aufgrund der starken Tiefpassfilterung der Akzelerometer benötigt der Filter eine gewisse Zeit, bis er den tatsächlichen Wert des Winkels einschwingt. Es ist daher ratsam, die Zustandsschätzung einige Sekunden vor dem Start der Regelung zu aktivieren, damit der Regler von Beginn an physikalisch korrekte Daten erhält.

5 Reglerauslegung und Simulation

Der Regler zur Stabilisierung des Würfels soll auf Basis des LQR-Ansatzes mit Matlab/Simulink ausgelegt werden. Anschließend soll der Regler in der Simulation validiert werden. Ausgangspunkt bildet die linearisierte Zustandsraumdarstellung des Systems, beschrieben durch die Matrizen A und B, die aus der Modellierung bekannt sind. Verwenden Sie das Modell ohne Reibung. Auf dieser Grundlage wird ein zeitdiskreter Zustandsregler mit einer Abtastperiode von $T = 20\,\mathrm{ms}$ entworfen. Für die initiale Auslegung des Reglers werden zunächst Einheitsgewichtungen verwendet:

$$Q = I, \qquad R = 1.$$

Zur Bewertung der Reglerauslegung wird der geschlossene Regelkreis an einem nichtlinearen Modell des Würfels untersucht. Hierfür wird der Zustandsregler in ein geeignetes Subsystem integriert, das mit der gewählten Abtastperiode arbeitet. Die Simulation erfolgt ausgehend von einer Anfangsauslenkung des Würfels von 10°, um das Einschwingverhalten und die Stabilisierung des Systems zu analysieren.

Im letzten Schritt wird der Zustandsregler gezielt abgestimmt, um ein gewünschtes dynamisches Verhalten zu erreichen. Ziel ist es, dass alle drei Zustandsgrößen innerhalb von etwa 3 s gegen Null konvergieren, während das aufgebrachte Drehmoment einen Betrag von ungefähr 0,5 Nm nicht überschreitet. Dazu werden die Gewichtungsmatrizen des LQR-Entwurfs angepasst. Es werden Diagonalmatrizen der Form

$$Q = \begin{pmatrix} q_1 & 0 & 0 \\ 0 & q_2 & 0 \\ 0 & 0 & q_3 \end{pmatrix}, \qquad R = r$$

verwendet, wobei die Parameter q_1, q_2, q_3 und r systematisch variiert werden, bis die gewünschte Regelgüte und Aktuatorbegrenzung erreicht sind.

Abbildungsverzeichnis

2.1 VSCode Oberfläche 5

3.1 Übersicht Software 7
3.2 Softwarearchtitektur 8

Quelltextverzeichnis

1 Clone VSCode-Repository 4
2 Aufbau zum Server der Würfel 5
3 Aufbau SSH Verbindung zum Würfel 6